
Nombre:

DNI / NIE:

Análisis y diseño de algoritmos
Examen final de la convocatoria junio (C3)
duración: 90 min.

Instrucciones:

- ANTES DE COMENZAR EL EXAMEN poned vuestros datos en el cuadernillo de preguntas y también en la hoja de respuestas.
- NO OLVIDES poner la modalidad en la hoja de respuestas.
- DEJAD encima de la mesa vuestro DNI o vuestra TIU.
- No podéis consultar ningún material ni hablar con nadie.
- Cada pregunta solo tiene una opción correcta, marcadla como se indica en la hoja de respuestas.
- Cada respuesta incorrecta resta la mitad de una correcta. Las preguntas sin contestar ni suman ni restan puntos.
- Tenéis 90 min. para hacer el examen.

Normativa: (Reglamento para la evaluación de los aprendizajes, 27-11-2015)

- Está prohibido acceder al aula del examen con cualquier tipo de dispositivo electrónico.
- Además de dos bolígrafos o lápices, del documento de identificación personal y del material suministrado por el profesorado, NO se permite tener ningún objeto o documento, ni en la mesa ni en sus inmediaciones.
- Si tienes dudas acerca de un objeto o dispositivo concreto pregunta al profesor antes de que comience la prueba.
- El incumplimiento de esta normativa puede conllevar, entre otras, la expulsión del aula del examen sin posibilidad de realizar la prueba.

Algunas preguntas hacen referencia al problema con el que hemos trabajado en las sesiones de prácticas. Su enunciado común es el siguiente:

Problema del laberinto: Se dispone de una cuadrícula $n \times m$ de valores $\{0, 1\}$ que representa un laberinto. Un valor 0 indica que la posición es inaccesible; por el contrario, con el valor 1 se simbolizan las casillas accesibles. Se pretende conocer la longitud del camino de salida más corto o un camino de salida con esa longitud (o ambas cosas).

Se define *longitud del camino* como el número de casillas que lo componen. Se define *camino de salida* como un camino que partiendo del origen del laberinto $(0, 0)$, conduce a la salida $(n - 1, m - 1)$. De este problema hemos distinguido dos versiones según los movimientos permitidos:

- **versión restringida:** Asumimos solo tres movimientos posibles: derecha; diagonal y abajo, siempre que la casilla de destino sea una posición válida.
- **versión general:** Desde una celda se puede acceder a cualquier de sus ocho colindantes siempre que la casilla destino sea una posición válida.

Modalidad 0

Preguntas:

1. Dada la siguiente función:

```
int exa (string & cad, int pri, int ult){

    if (pri>=ult)
        return 1;
    else
        if (cad[pri]==cad[ult])
            return exa(cad, pri+1, ult-1);
        else
            return 0;
}
```

¿Cuál es su complejidad temporal asintótica en función de $n = \text{ult} - \text{pri} + 1$?

- (a) $O(\log n)$
 - (b) $O(n^2)$
 - (c) $O(n)$
2. Al resolver el problema del “Viajante de comercio” mediante vuelta atrás, ¿cuál de estas cotas optimistas se espera que puede mejorar el árbol de búsqueda?
- (a) Se ordenan las aristas restantes de menor a mayor distancia y se calcula la suma de las k aristas más cortas, donde k es el número de saltos que nos quedan por dar.
 - (b) Se multiplica k por la distancia de la arista más corta que nos queda por considerar, donde k es el número de saltos que nos quedan por dar.
 - (c) Se resuelve el resto del problema usando un algoritmo voraz que añade cada vez al camino el vértice más cercano al último añadido.
- 3.Cuál de las siguientes formulaciones expresa mejor el coste temporal asintótico de la siguiente función?

```
int f(int n) {
int count = 0;
for (int i = n; i > 0; i /= 2)
    for (int j = 0; j < 2 * i; j++)
        count += 1;
return count;
}
```

- (a) $f(n) = \sum_{i=1}^{\log n} 4n \left(\frac{1}{2}\right)^i$
 - (b) $f(n) = \sum_{i=0}^{n/2} \sum_{j=0}^{2*i} 1$
 - (c) Ninguna de las otras dos opciones es correcta
4. Con respecto a la versión general (sin restricciones adicionales) del problema de la mochila sin fraccionamiento, de las siguientes afirmaciones dos son verdaderas y una es falsa, o viceversa. Señala la que difiere de las otras dos.
- (a) ... cumple la propiedad de subestructura óptima y por lo tanto se puede plantear una solución de divide y vencerás
 - (b) ... no se conoce ninguna reducción en subproblemas que cumpla el teorema de reducción
 - (c) ... tiene una solución eficiente utilizando programación dinámica

5. En un algoritmo de vuelta atrás, ¿se podrían priorizar las expansiones de los nodos con mejor cota optimista de manera que el algoritmo siga siendo de vuelta atrás?
- (a) No, ya no sería de vuelta atrás
 - (b) Sí, pero a costa de aumentar considerablemente su complejidad temporal
 - (c) Sí, basta con cambiar la estrategia del búsqueda
6. ¿Cuál es el propósito principal de un *Heap* en el algoritmo de ordenación *HeapSort*?
- (a) Permitir la extracción repetida del elemento máximo o mínimo de manera eficiente
 - (b) Lograr una ordenación parcial del vector, es decir, que algunos elementos queden en la posición correcta pero otros no.
 - (c) Facilitar la inserción eficiente de elementos en el nuevo vector ordenado.
7. Si un algoritmo de ramificación y poda utiliza una cola de prioridad con un criterio inadecuado, ¿qué ocurrirá muy probablemente?
- (a) Aumentará considerablemente el número de instancias que están en el caso peor.
 - (b) Seguirá siendo más eficiente que la vuelta atrás gracias al uso de la cola de prioridad.
 - (c) Las otras dos opciones son correctas.
8. Si el coste temporal de un algoritmo es $T(n)$, ¿cuál de las siguientes situaciones es imposible?
- (a) $T(n) \in O(n)$ y $T(n) \in \Theta(n)$
 - (b) $T(n) \in \Omega(n)$ y $T(n) \in \Theta(n^2)$
 - (c) $T(n) \in \Theta(n)$ y $T(n) \in \Omega(n^2)$
9. Asumiendo un modelo de computación en el que los números enteros pueden ser arbitrariamente grandes ¿Cuál es la complejidad temporal asintótica de la siguiente función (en función del tamaño del problema)?
- ```
unsigned factorial(unsigned n) {
 if (n == 0 || n == 1)
 return 1;
 else
 return n * factorial(n - 1);
}
```
- (a) Ninguna de las otras dos opciones es cierta
  - (b)  $\Theta(n)$
  - (c)  $\Theta(\log n)$
10. Decid cuál de estas tres afirmaciones es cierta
- (a) El valor óptimo de la mochila discreta es una cota superior del valor óptimo de la mochila continua correspondiente
  - (b) El valor óptimo de la mochila continua es una cota superior del valor óptimo de la mochila discreta correspondiente
  - (c) las otras dos afirmaciones son falsas

11. Estamos resolviendo el problema del laberinto (versión general) mediante ramificación y poda empleando una estrategia dirigida de priorización de nodos basada en una cola de prioridad. Recuerda que cada iteración del bucle principal consiste en extraer el nodo de máxima prioridad de la cola, expandirlo, y, para cada nodo expandido: comprobar si es factible, actualizar la solución en curso con la cota pesimista parcial, y añadirlo a la cola si es prometedor. La mejor complejidad asintótica que se puede conseguir para una iteración del bucle principal es:
- (a)  $O(1)$ .
  - (b)  $O(\log n)$ , siendo  $n$  el número de nodos en la cola.
  - (c)  $O(n)$ , siendo  $n$  el número de nodos en la cola.
12. En el algoritmo de Kruskal, ¿qué se selecciona en cada paso?
- (a) El vértice con menor grado en el grafo
  - (b) La arista de menor peso que conecta un vértice visitado con otro sin visitar
  - (c) La arista de menor peso que no forma ciclos
13. Se pretende obtener la complejidad temporal en el caso más desfavorable de la siguiente función.

```
int exa (vector <int>& v) {
 int i, sum=0, n=v.size();

 if (n>0){
 int j=n;
 while (j>0 && sum<100){
 j=j/2;
 sum=0;
 for (i=j; i<n; i++)
 sum+=v[i];
 }
 return j;
 }
 else return -1;
}
```

¿Cuál de las siguientes formulaciones expresa dicho coste?

- (a)  $c_s(n) = n \sum_{j=1}^{\log n} \sum_{i=1}^j \left(\frac{1}{2}\right)^i \in O(n \log n)$
  - (b) Las otras dos opciones son ciertas
  - (c)  $c_s(n) = \sum_{k=1}^{\log n + 1} (n - n/2^k) \in O(n \log n)$
14. ¿Cuál es la complejidad temporal de la aproximación voraz a la solución de la versión restringida del problema del laberinto?
- (a)  $O(n + m)$
  - (b) Las otras dos opciones son ambas ciertas
  - (c)  $O(\max\{n, m\})$
15. Respecto a los algoritmos de vuelta atrás, ¿cuál de las siguientes afirmaciones es correcta?
- (a) Siempre encuentran una solución, aunque no siempre sea óptima.
  - (b) Siempre hallan la mejor solución, aunque requieren mucho tiempo al explorar todas las posibilidades.
  - (c) Las otras dos afirmaciones son falsas.

16. Dado el siguiente programa recursivo:

```
int f(unsigned n) {
 if(n == 0 || n == 1)
 return n;
 return f(n-1) + f(n-2);
}
```

si quisiéramos reescribirlo para usar programación dinámica ¿cuál la complejidad espacial más ajustada del algoritmo?

- (a)  $O(1)$
  - (b)  $O(n^2)$
  - (c)  $O(n)$
17. Estamos resolviendo el problema del laberinto mediante un algoritmo de búsqueda y enumeración. Supongamos que el mejor camino de salida encontrado hasta el momento es de longitud 100. Más adelante, tenemos que decidir acerca de un nodo  $n$  del que hemos obtenido una cota optimista que ha resultado ser 80 y una pesimista con valor 90. ¿Cuál de las siguientes afirmaciones es siempre cierta?
- (a) Algo está mal pues la cota pesimista recién calculada no puede ser inferior a 100
  - (b) Algo está mal pues la cota optimista recién calculada no puede ser inferior a 100
  - (c) El nodo  $n$  podría formar parte de la solución y además, tenemos la certeza de que su cota pesimista es la mejor encontrada hasta el momento
18. En la resolución del problema de la función compuesta mínima mediante vuelta atrás, ¿qué ocurriría si no se limitara la profundidad del árbol de llamadas recursivas con valor máximo  $M$ ?
- (a) El programa acabaría encontrando la solución óptima, pero a costa de una complejidad temporal prohibitiva.
  - (b) Se podría conseguir que el programa siempre encontrara solución poniendo un 1 en el vector solución antes que un 0.
  - (c) Salvo en casos muy raros, el programa nunca acabaría.
19. Cuál es el coste temporal asintótico de la siguiente función?

```
int f(int n) {
 int count = 0;
 for (int i = n; i > 0; i -= 2)
 for (int j = i; j > 0 ; j /= 2)
 count += 1;
 return count;
}
```

- (a)  $O(n)$
  - (b)  $O(n \log n)$
  - (c)  $O(\log^2 n)$
20. Dada la versión restringida del problema del laberinto  $m \times n$ , estamos interesados, no solo en obtener la longitud del camino de salida más corto; también nos interesa la longitud del camino más corto entre el origen y cualquier casilla accesible del laberinto ¿Con qué complejidad temporal, en función de  $n$  y  $m$ , podría obtenerse?
- (a)  $\Theta(m \cdot n)$
  - (b) Ninguna de las otras dos opciones es correcta.
  - (c)  $\Theta(3^{(m+n)})$

21. Respecto a los algoritmos voraces, ¿cuál de las siguientes afirmaciones es correcta?
- (a) Siempre encuentran una solución, aunque no siempre sea óptima.
  - (b) Siempre hallan la mejor solución, aunque requieren mucho tiempo al explorar todas las posibilidades.
  - (c) Las otras dos afirmaciones son falsas.
22. Sobre el esquema de programación dinámica, dos afirmaciones son verdaderas y una falsa, o viceversa. Señala la que difiere de las otras dos.
- (a) Se apoya en el esquema de divide y vencerás, siendo una mejora de éste en términos de eficiencia.
  - (b) Es recomendable, aunque no imprescindible, que el problema cumpla con la condición de estructura óptima.
  - (c) Solo se refiere a la conversión de un algoritmo recursivo en iterativo para mejorar su eficiencia.
23. Asumiendo un modelo de computación en el que el número de bits utilizados para representar enteros está acotado, ¿Cuál es la complejidad temporal asintótica de la siguiente función (en función del tamaño del problema)?
- ```
unsigned factorial(unsigned n) {
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}
```
- (a) En este caso no tiene sentido hablar de complejidad temporal asintótica
 - (b) $\Theta(n)$
 - (c) $\Theta(\log n)$
24. La búsqueda binaria puede considerarse una instancia del esquema:
- (a) Divide y vencerás.
 - (b) Programación dinámica.
 - (c) Algoritmo voraz.
25. En la resolución del problema del laberinto (versión general), el uso de una cota pesimista parcial sirve para:
- (a) Evitar ciclos.
 - (b) Descartar nodos cuando se llega a una posición del mapa mediante un camino más largo que el mejor camino para llegar a esa posición.
 - (c) Mejorar la solución en curso antes de llegar a una hoja.
26. Indica cuál de estos criterios para ordenar la lista de nodos vivos permite resolver más rápido el problema de la función compuesta mínima:
- (a) Búsqueda en anchura.
 - (b) Búsqueda en profundidad.
 - (c) Ambos criterios son válidos y no afectan al tiempo de ejecución.
27. Respecto a la estrategia de ramificación y poda...
- (a) Siempre produce algoritmos más eficientes que la de vuelta atrás.
 - (b) La principal diferencia con la de vuelta atrás es el uso de estrategias *Least Cost*.
 - (c) Las otras dos afirmaciones son ciertas.

28. Dada la versión restringida del problema del laberinto resuelto mediante programación dinámica iterativa comenzando en la entrada al laberinto. ¿Qué representa el contenido de cada casilla (i, j) del almacén?
- La longitud del camino más corto desde esa casilla hasta el destino.
 - La longitud del camino más corto que llega a esa casilla desde el origen.
 - La longitud del camino más corto desde esa casilla hasta el destino pero solo si la casilla forma parte del camino óptimo.
29. Estamos resolviendo el problema del laberinto mediante un algoritmo de búsqueda y enumeración. Supongamos que nos encontramos en un nodo interno n situado en un nivel x del árbol de estados posibles. ¿Qué podemos decir acerca del valor de x ?
- Es una cota pesimista de n
 - Es una cota optimista de n
 - Por sí solo, x no es cota, ni optimista ni pesimista
30. En cuanto al problema del cambio, que consiste en formar una suma M con el mínimo número de monedas posibles, extraídas (con repetición) de un conjunto C :
- Es posible diseñar una solución basada en el esquema divide y vencerás que siempre encuentra el valor óptimo.
 - Es posible diseñar una solución voraz que siempre encuentra el valor óptimo.
 - Las otras dos opciones son correctas.
31. Dada la versión general del problema del laberinto $m \times n$, la *distancia de Manhattan* entre las casillas (i, j) y (m, n) viene dada por la suma del número movimientos del tipo horizontal que quedan hasta llegar al extremo derecho del laberinto $(n - j)$ y los del tipo vertical que quedan hasta llegar al extremo inferior del laberinto $(m - i)$. ¿Para qué puede resultar útil esta distancia? ($d_{\text{manhattan}}(i, j) = n - j + m - i$)
- Para obtener cotas optimistas
 - Para obtener cotas pesimistas
 - Ninguna de las otras dos opciones es válida
32. Indica cuál de estas posibles cotas pesimistas para el problema del laberinto (versión general) sería la más ajustada:
- La longitud del camino actual más la de su continuación hasta el final del mapa mediante un algoritmo voraz que sólo considera movimientos hacia E, SE o S.
 - La longitud del camino actual.
 - La longitud del camino actual más la de su continuación hasta el final del mapa mediante un algoritmo de programación dinámica que sólo considera movimientos hacia E, SE o S.
33. En la resolución del problema de corte de tubos mediante programación dinámica:
- La programación dinámica recursiva (memoización) siempre resuelve menos subproblemas que la programación dinámica iterativa.
 - La programación dinámica recursiva (memoización), dependiendo de la instancia del problema, puede resolver menos subproblemas que la programación dinámica iterativa.
 - La programación dinámica recursiva (memoización) y la programación dinámica iterativa siempre resuelven los mismos subproblemas.
34. Disponemos de dos funciones de coste $f(n)$ y $g(n)$ tal que $f \in O(g)$. ¿Puede existir un valor x para el que $f(x) > g(x)$?
- Sí, pero también existirá un valor n_0 tal que $\forall_{n \geq n_0} f(n) \leq g(n)$.
 - Sí, y además es posible que el valor n_0 tal que $\forall_{n \geq n_0} f(n) \leq g(n)$ no exista.
 - No, no puede existir ese valor x .

35. Di cuál de estos resultados de coste temporal asintótico es falsa:
- (a) La ordenación de un vector usando el algoritmo *Quicksort* requiere en el peor caso un tiempo en $\Omega(n^2)$.
 - (b) La ordenación de un vector usando el algoritmo *Mergesort* requiere en el peor caso un tiempo en $\Omega(n^2)$.
 - (c) La búsqueda binaria en un vector ordenado requiere en el peor caso un tiempo en $O(\log n)$.
36. Sea $g(n) = \sum_{j=0}^{\log_2 n} n4^j$. Suponiendo $n \rightarrow \infty$, di cuál de las siguientes afirmaciones es cierta:
- (a) $g(n) \in O(n^2 \log n^2)$
 - (b) $g(n) \in \Omega(n^2)$
 - (c) $g(n) \in \Theta(n^2)$
37. En un problema de minimización mediante ramificación y poda:
- (a) La cota pesimista parcial de un nodo tiene que ser siempre mayor o igual que la cota pesimista del nodo inicial.
 - (b) La cota pesimista parcial de un nodo tiene que ser siempre menor o igual que la mejor solución en curso.
 - (c) Ninguna de las otras dos opciones es correcta.
38. Sobre el problema del “Viajante de comercio” resuelto mediante vuelta atrás, dos afirmaciones son verdaderas y una falsa, o viceversa. Señala la que difiere de las otras dos.
- (a) Aún con una implementación sin cotas, es un problema con complejidad temporal abordable.
 - (b) El cálculo incremental del recorrido y el uso de cotas (pesimistas y optimistas) reducen notablemente la complejidad temporal en el peor caso.
 - (c) Las otras dos afirmaciones son falsas.
39. En un algoritmo de ramificación y poda, conforme se va completando un nodo, ¿qué ocurre con sus cotas?
- (a) Su cota pesimista debería aproximarse a la mejor solución que puede obtenerse a partir de ese nodo.
 - (b) Su cota optimista debería aproximarse a la mejor solución que puede obtenerse a partir de ese nodo.
 - (c) Las otras dos afirmaciones son ciertas.
40. La solución voraz del problema de la mochila continua, es decir, con objetos fraccionables, tiene una complejidad temporal asintótica de:
- (a) $O(\log n)$
 - (b) $O(n)$
 - (c) $O(n \log n)$